



NGUYÊN LÝ HỆ ĐIỀU HÀNH

Phần 3: Tiến trình

NGUYỄN THỊ HẬU

Khoa Công Nghệ Thông Tin
Đại học Công Nghệ - Đại học quốc gia Hà Nội

Phần 3: Tiến trình

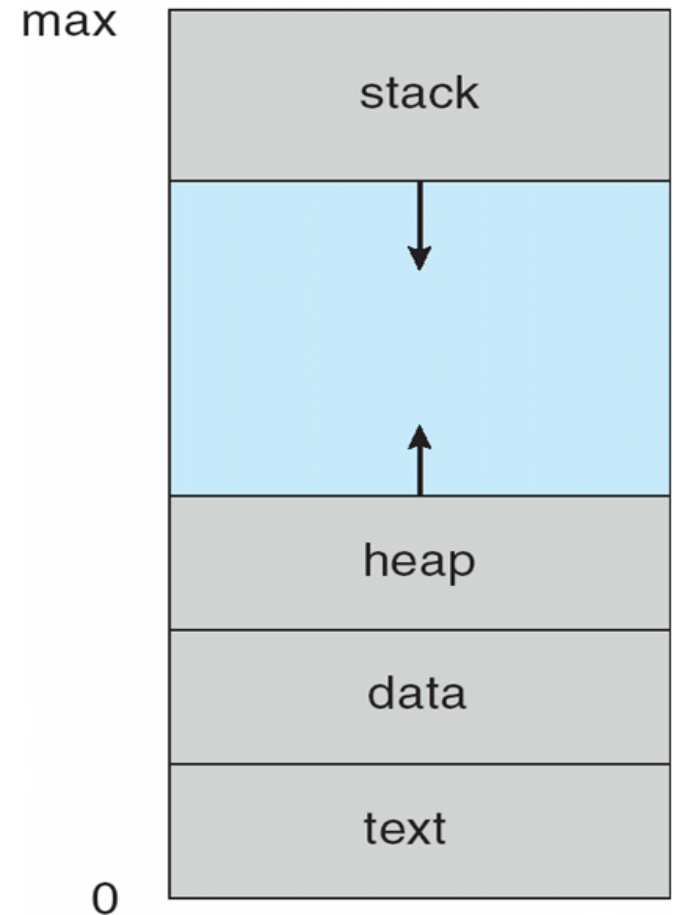
- Khái niệm
- Lập lịch tiến trình
- Thao tác với tiến trình
- Truyền thông giữa các tiến trình

Phần 3: Tiến trình

3.1 Khái niệm tiến trình

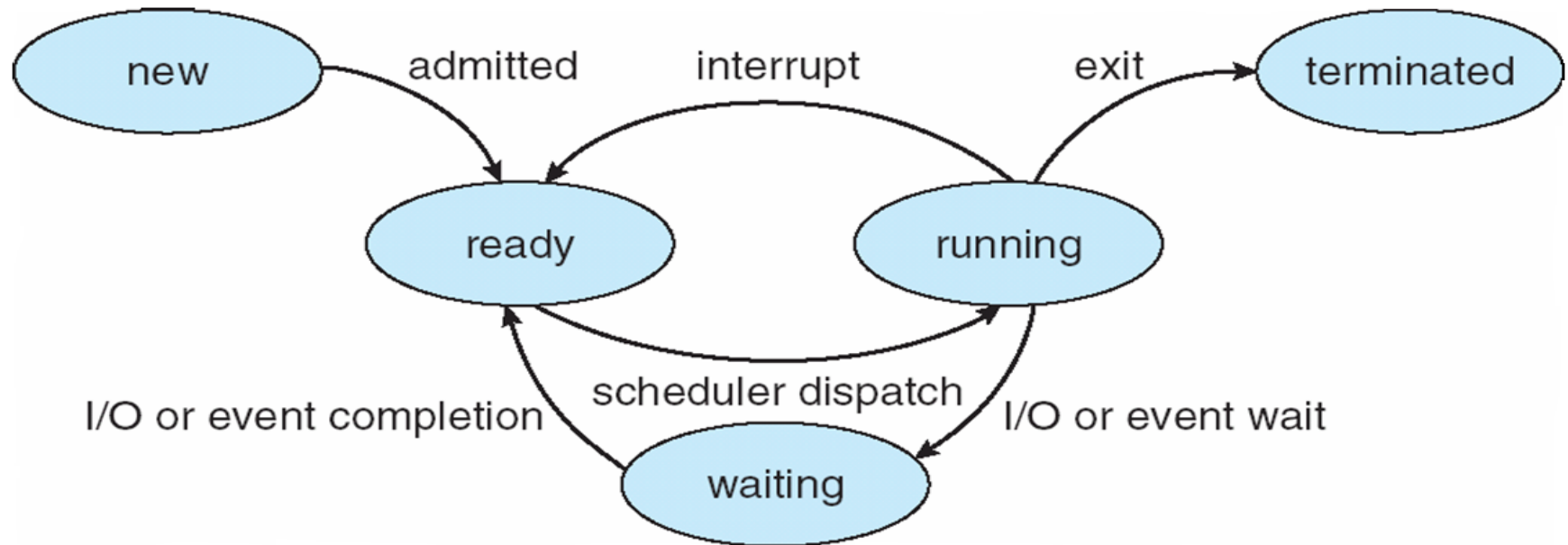
Tiến trình là gì ?

- Thuật ngữ: **process**
- **Tiến trình** là một chương trình đang được thực thi
- Một tiến trình bao gồm:
 - Phần văn bản chứa mã chương trình
 - Bộ đếm của chương trình
 - Ngăn xếp chứa dữ liệu tạm thời
 - Phần dữ liệu chứa biến toàn cục
 - Heap chứa bộ nhớ động cấp phát trong lúc chạy



Các trạng thái của tiến trình

5



Khối điều khiển tiến trình (Process Control block - PCB)

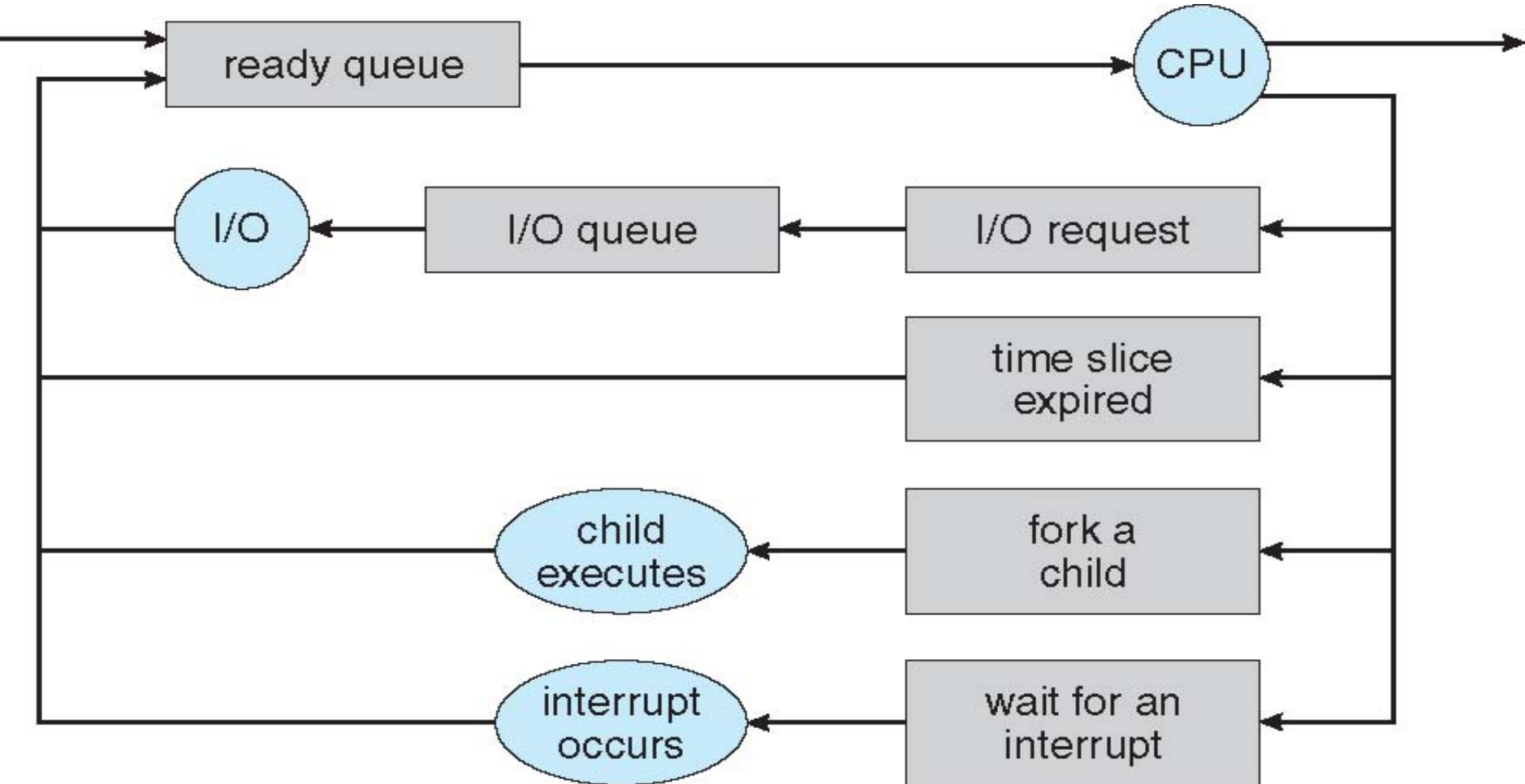
Con trỏ	Trạng thái
Số thứ tự	
Bộ đếm	
Thanh ghi	
Giới hạn bộ nhớ	
Danh sách các tệp tin mở	
.....	

Phần 3: Tiến trình

3.2 Lập lịch tiến trình

- Nhằm tối ưu việc sử dụng CPU trong hệ thống cho phép chia sẻ thời gian
- **Hàng chờ lập lịch** (scheduling queues)
 - **Hàng chờ nhiệm vụ** (job queue): bao gồm các tiến trình trong hệ thống
 - **Hàng chờ sẵn sàng** (Ready queue): bao gồm các tiến trình đang ở bộ nhớ chính, sẵn sàng và chờ thực thi
 - **Hàng chờ thiết bị** (Device queues): bao gồm các tiến trình đang chờ một thiết bị vào /ra
 - Các tiến trình di chuyển giữa các hàng chờ trên

Hàng chờ lập lịch tiến trình

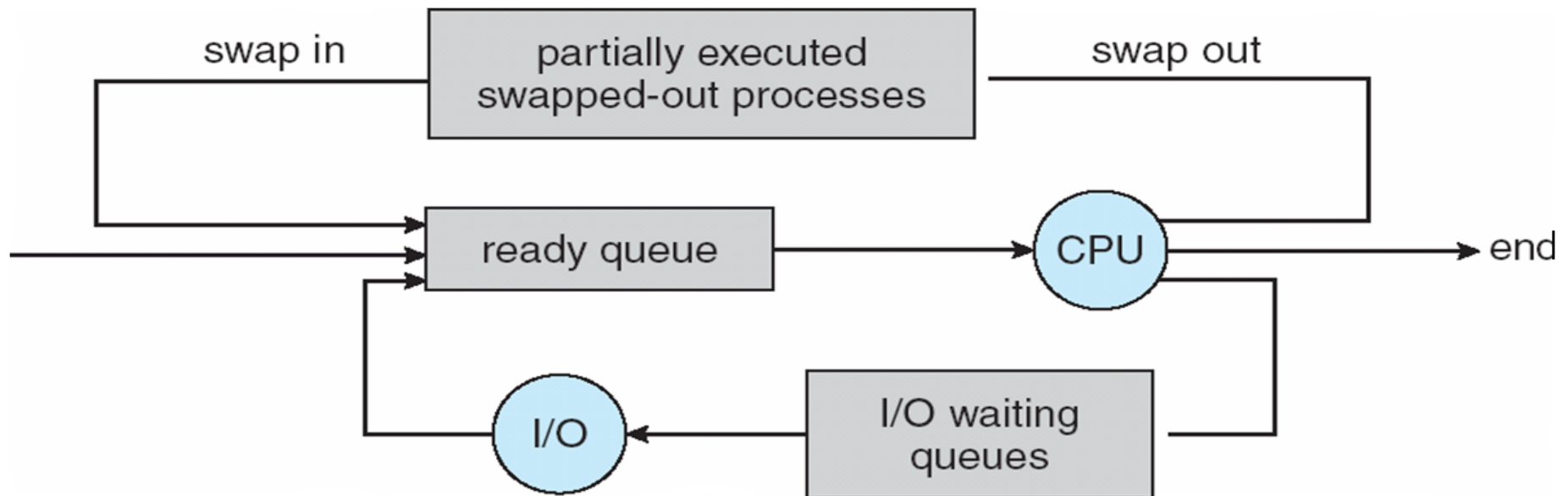


Phân loại các bộ lập lịch

- *Bộ lập lịch dài hạn* (long-term scheduler/job scheduler)
 - Lựa chọn tiến trình để chuyển sang hàng chờ sẵn sàng
 - Đóng vai trò điều khiển cấp độ của đa chương trình
- *Bộ lập lịch ngắn hạn* (short-term scheduler/CPU scheduler)
 - Lựa chọn tiến trình tiếp theo được sử dụng CPU
- *Bộ lập lịch trung hạn* (medium-term scheduler/swapping)
 - Di chuyển tiến trình đang trong trạng thái chờ giữa bộ nhớ trong và bộ nhớ ngoài

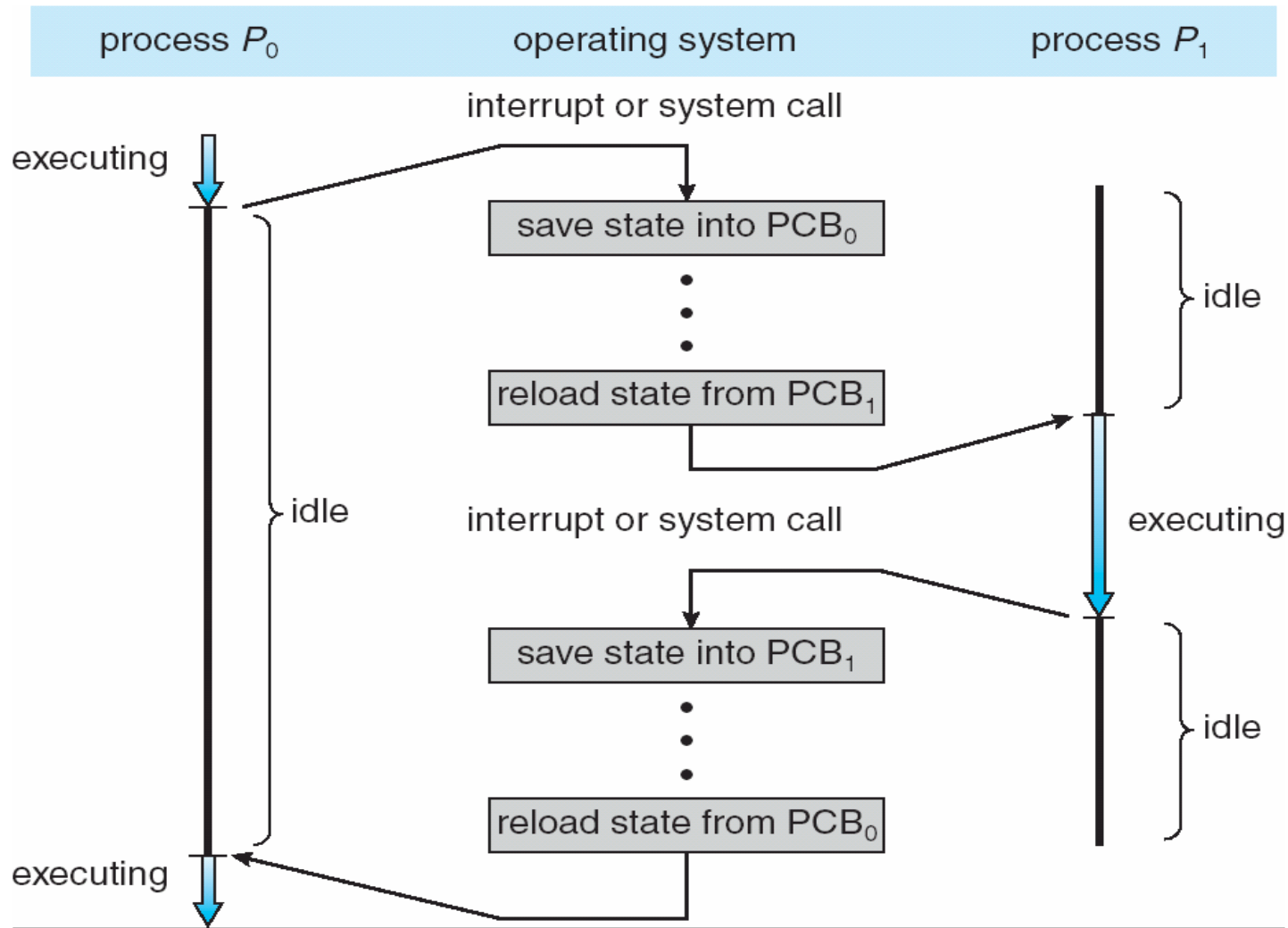
Bộ lập lịch trung hạn

11



CPU chuyển từ tiến trình này sang tiến trình khác

12



Chuyển trạng thái (Context Switch)

13

- Được thực hiện khi CPU chuyển giữa các tiến trình
- Trạng thái của một tiến trình được lưu trong khối điều khiển tiến trình (PCB)
- Khi thực hiện chuyển trạng thái, hệ thống không làm việc gì khác
- Thời gian thực hiện chuyển trạng thái tùy thuộc vào độ phức tạp của HĐH và khối điều khiển tiến trình

Phần 3: Tiến trình

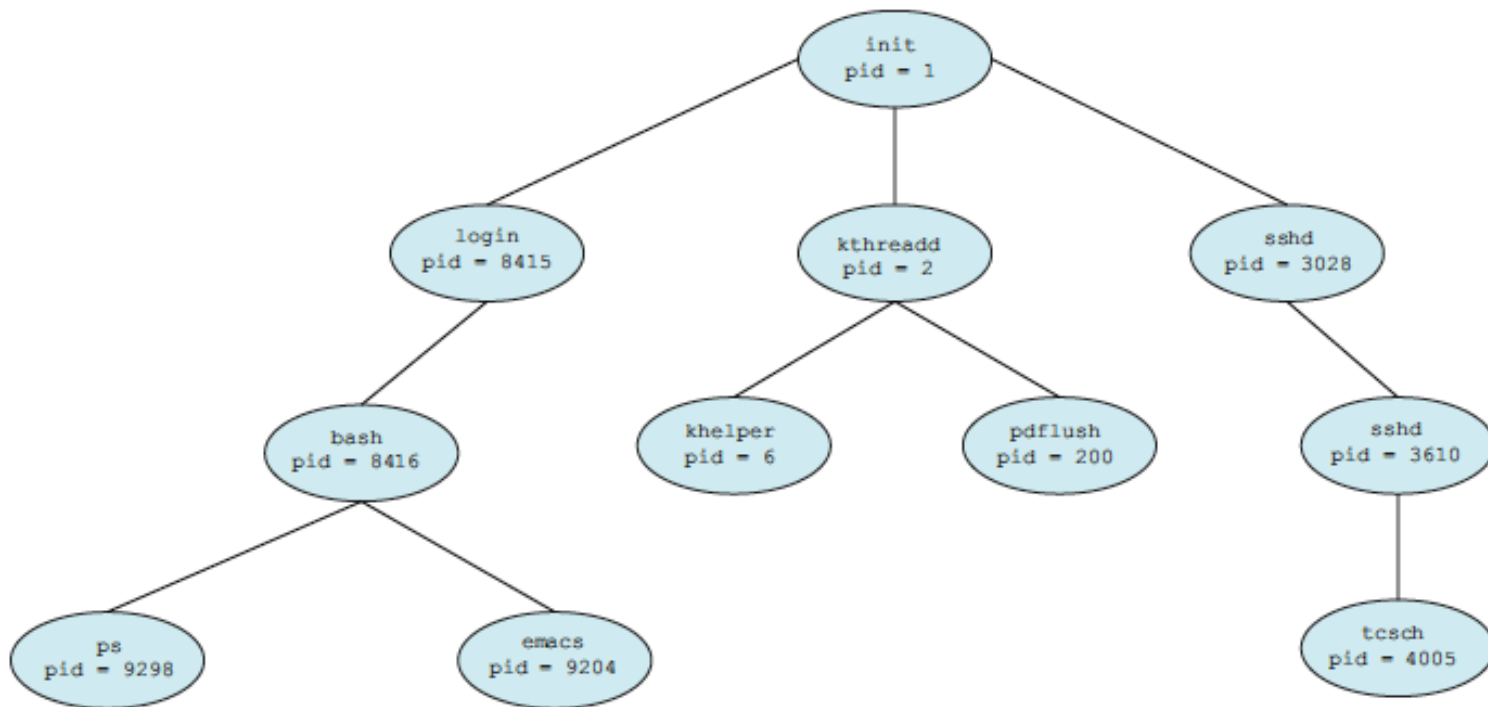
3.2 Các thao tác với tiến trình

- Có bốn cách để tạo tiến trình:
 - Khởi tạo hệ thống
 - Một tiến trình đang chạy thực thi hàm hệ thống tạo tiến trình
 - Người dùng yêu cầu
 - Bắt đầu thực thi một nhiệm vụ trong chế độ xử lý theo lô
- Mỗi tiến trình có một số hiệu tiến trình (process identifier – pid)
- Thực thi (có 2 khả năng):
 - Tiến trình cha và con thực thi đồng thời
 - Tiến trình cha đợi đến khi các tiến trình con kết thúc

Cây tiến trình

16

- Mỗi **tiến trình cha** có thể tạo một số **tiến trình con**, và những tiến trình con này có thể tạo các tiến trình con khác, tạo thành **cây tiến trình**



Ví dụ tạo tiến trình trong UNIX

17

```
#include <sys/types.h>
#include <sys/wait.h>
#include <stdio.h>
#include <unistd.h>

int value = 5;
int main(){
    printf("\n Before fork()");
    pid_t pid;
    pid = fork();

    if (pid < 0) { /* error occurred */
        fprintf(stderr, "\n Fork Failed");
        return 1;
    }

    else if (pid == 0) { /* child process */
        value += 15;
        printf("\n CHILD: value = %d",value);
        /* LINE A */
        return 0;
    }
    else if (pid > 0) { /* parent process */
        wait(NULL);
        printf("\n PARENT: value = %d",value);
        /* LINE B */
        return 0;
    }
}
```

```
$ gcc myfork.c -o myfork
$ ./myfork
```

Ví dụ tạo tiến trình trong UNIX

18

Before fork()

CHILD: value = 20

Before fork()

PARENT: value = 5

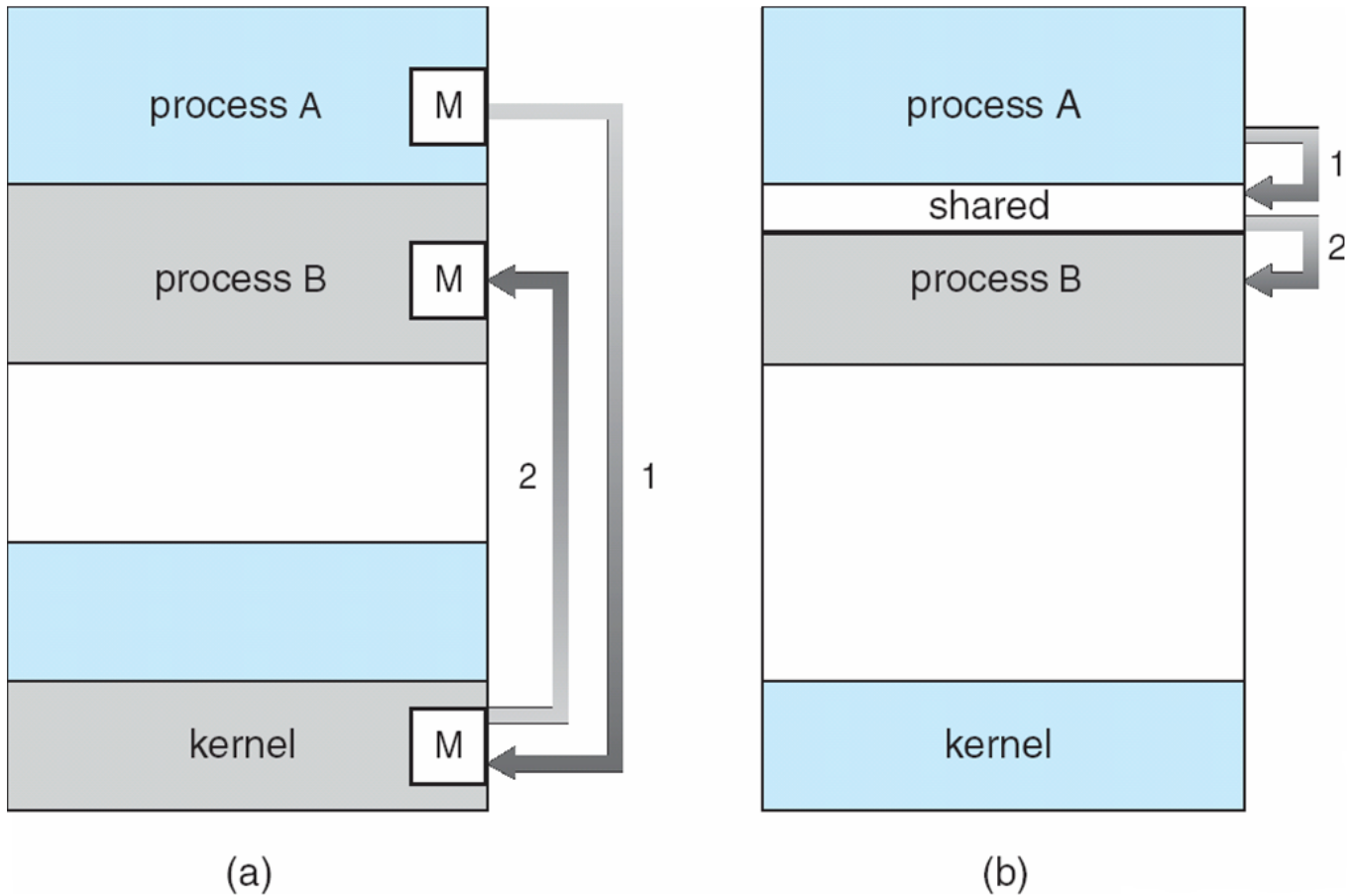
- Tiến trình thực thi câu lệnh cuối cùng và yêu cầu HĐH xoá nó (**exit**)
 - Chuyển dữ liệu từ tiến trình con sang tiến trình cha (thông qua **wait**)
 - Các tài nguyên của tiến trình được trả lại cho HĐH
- Tiến trình cha có thể kết thúc việc thực thi của các tiến trình con (thông qua **abort/kill**)
 - Khi tiến trình con sử dụng vượt qua nguồn tài nguyên đã được cấp phát
 - Nhiệm vụ giao cho tiến trình con không cần thiết nữa
 - Nếu tiến trình cha kết thúc, một số HĐH không cho phép các tiến trình con được tiếp tục thực thi

Phần 3: Tiến trình

3.2 Truyền thông giữa các tiến trình

- Các tiến trình trong một hệ thống có thể là tiến trình *độc lập* hoặc *hợp tác*
- Tiến trình hợp tác có thể ảnh hưởng đến / bị ảnh hưởng bởi tiến trình khác, bao gồm việc chia sẻ dữ liệu
- Mục đích của việc hợp tác tiến trình:
 - Chia sẻ thông tin
 - Tính toán nhanh hơn
 - Cho phép mô đun hoá
 - Thuận tiện
- Hợp tác tiến trình cần phương thức truyền thông giữa các tiến trình (IPC)

Các mô hình truyền thông



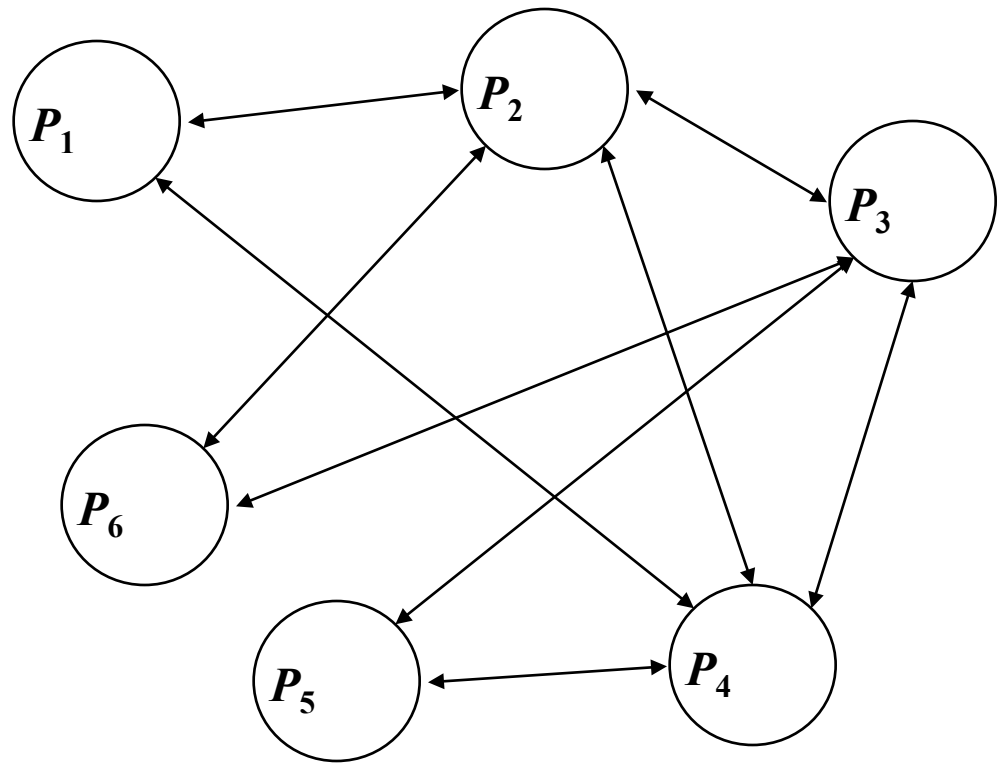
(a) Mô hình trao đổi thông điệp (b) Mô hình chia sẻ bộ nhớ

Các hệ thống truyền thông điệp

- Cho phép các tiến trình truyền thông với nhau qua các toán tử **send()** và **receive()**
- Cần có một kết nối giữa tiến trình P và Q để truyền thông điệp:
 - *Kết nối vật lý*: chia sẻ bộ nhớ, đường truyền phần cứng
 - *Kết nối logic*: các thuộc tính logic
- Một số loại truyền thông điệp qua kết nối logic:
 - Trực tiếp hoặc gián tiếp
 - Đối xứng hoặc không đối xứng
 - Sử dụng vùng đệm tự động hoặc không tự động

Truyền thông trực tiếp đối xứng

- Hai tiến trình phải gọi rõ tên của nhau
 - **send**(P, msg): Gửi msg đến tiến trình P
 - **receive**(Q, msg): Nhận msg từ tiến trình Q
- Chỉ duy nhất một kết nối được thiết lập tự động giữa hai tiến trình
- Kết nối một hướng hoặc hai hướng



Truyền thông trực tiếp không đối xứng

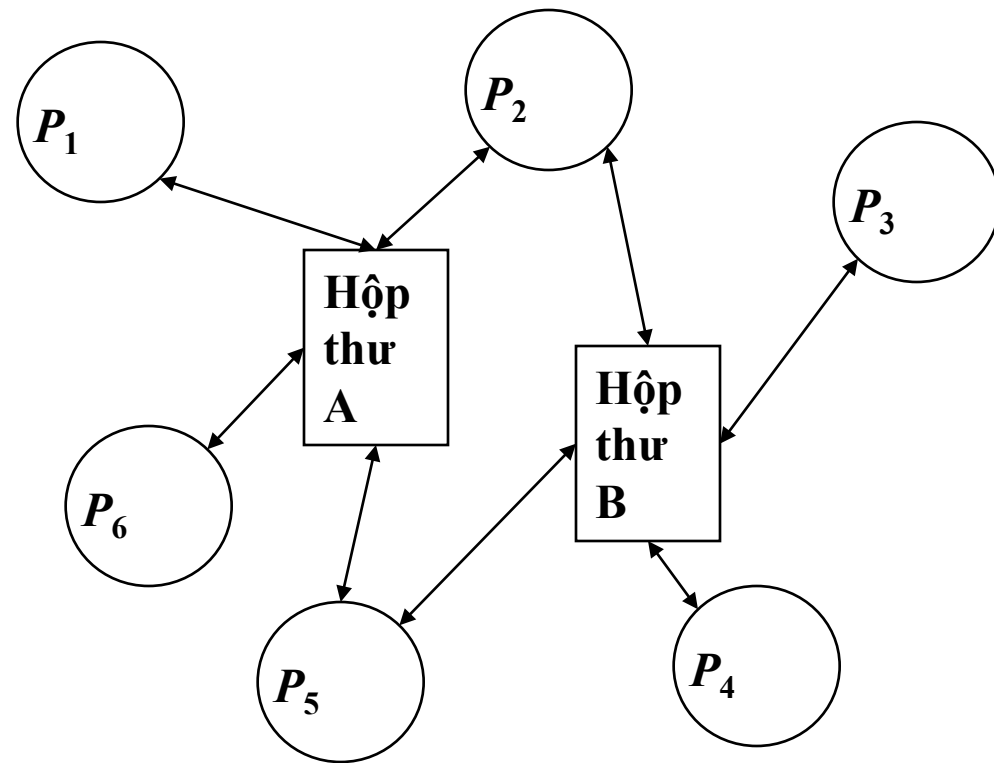
- Chỉ tiến trình gửi gọi tên tiến trình nhận
 - **send**(P, msg): Gửi msg đến tiến trình P
 - **Receive**(id, msg): Tiến trình id nhận msg từ bất cứ tiến trình nào
- **Nhược điểm của truyền thông trực tiếp:**
 - Cần biết trước id của tiến trình gửi/nhận
 - Thay đổi id của tiến trình gửi/nhận sẽ ảnh hưởng đến quá trình truyền thông

- Các thông điệp được gửi và nhận qua các *hộp thư* (mailbox) hoặc qua các *cổng* (port)
- Hộp thư do HĐH hoặc một tiến trình tạo ra
- Hai toán tử:
 - **send**(A , msg): Gửi msg đến hộp thư A
 - **receive**(A , msg): Nhận msg từ hộp thư A

Minh họa truyền thông gián tiếp

27

- Hai tiến trình có kết nối nếu sử dụng chung một hộp thư
- Một kết nối có thể sử dụng cho nhiều tiến trình (≥ 2)
- Nhiều kết nối có thể tồn tại giữa một cặp tiến trình (nếu sử dụng các hộp thư khác nhau)



Vấn đề đồng bộ hóa (Synchronization)

- Việc trao đổi thông điệp có thể ở **phương thức có chờ** (blocking) **hoặc phương thức không chờ** (non-blocking)
- Phương thức chờ được coi là đồng bộ:
 - Chế độ chờ gửi: tiến trình gửi chờ đến khi thông điệp được nhận
 - Chế độ chờ nhận: tiến trình nhận chờ đến khi có thông điệp
- Phương thức không chờ được coi là không đồng bộ
 - Chế độ không chờ gửi: tiến trình gửi thông điệp rồi tiếp tục
 - Chế độ không chờ nhận: tiến trình nhận nhận một thông điệp hoặc không gì cả

Vấn đề sử dụng vùng đệm

- Các thông điệp nằm trong hàng chờ tạm thời
- Kích cỡ của hàng chờ:
 - Chứa được 0 thông điệp: **send** blocking
 - Chứa được n thông điệp: **send** non-blocking cho đến khi hàng chờ có n thông điệp, sau đó **send** blocking
 - Vô hạn: **send** non-blocking

- Lý thuyết:
 - Luồng (Thread)
 - Pipe
 - Tín hiệu (signal)

Bài tập

1. Bài toán **Bữa ăn của các nhà triết học** :

Có 5 nhà triết học ngồi trong một chiếc bàn tròn. Mỗi nhà triết học có một đĩa spaghetti. Vì spaghetti rất trơn nên mỗi nhà triết học phải dùng 2 chiếc nĩa để ăn. Giữa hai đĩa spaghetti chỉ có một chiếc nĩa. Vòng đời của nhà triết học là một chuỗi liên tiếp ăn và nghĩ. Khi nhà triết học đói, họ sẽ lấy nĩa bên trái và phải của mình cùng lúc. Nếu thành công, họ sẽ ăn một lúc rồi để nĩa xuống, rồi tiếp tục nghĩ.

Yêu cầu: Viết chương trình mô phỏng hành động các nhà triết học trên sao cho không xảy ra tình huống bế tắc.

Bài tập

2. Bài toán **Nhà sản xuất – Người dùng**:

Hai tiến trình chia sẻ một vùng đệm có kích thước cố định, trao đổi với nhau qua phương thức trao đổi thông điệp. Nhà sản xuất muốn đưa thông tin lên vùng đệm, còn người dùng muốn lấy thông tin từ vùng đệm.

Giả định tình huống vùng đệm đầy mà nhà sản xuất vẫn muốn đưa thông tin lên. Vậy nhà sản xuất phải chuyển sang trạng thái ngủ, được đánh thức bởi người dùng khi họ lấy thông tin từ vùng đệm. Tương tự, khi vùng đệm trống và người dùng muốn lấy thông tin từ vùng đệm, thì người dùng sẽ chuyển sang trạng thái ngủ và được đánh thức bởi nhà sản xuất khi họ đưa thông tin lên vùng đệm.

Yêu cầu: Viết chương trình minh họa cho tình huống trên